CPU Scheduling

Youngjae Kim (Ph.D.)
Sogang University
Linux 운영체제 및 응용 (GITF315-01)

Scheduling: Introduction

- **Workload assumptions:**
  1. Each job runs for the **same amount of time.**
  2. All jobs **arrive** at the same time.
  3. All jobs only use the **CPU** (i.e., they perform no I/O).
  4. The **run-time** of each job is known.

Scheduling Metrics

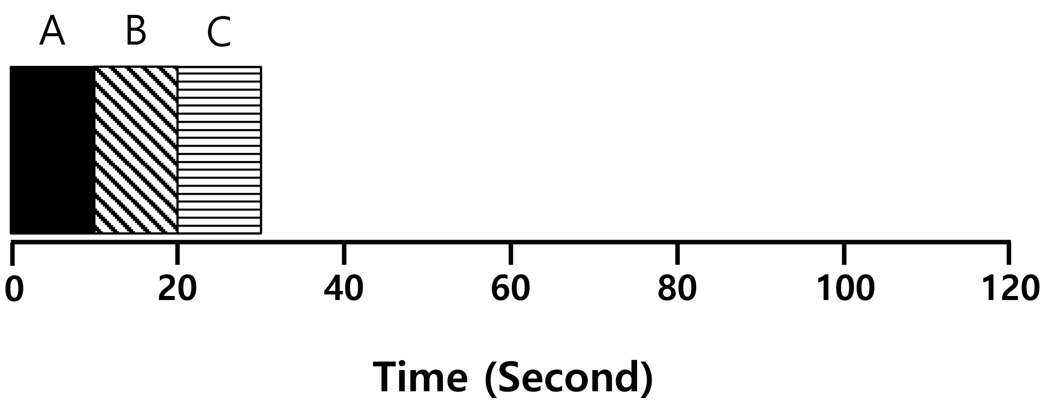
- **Performance metric: Turnaround time**
  - The time at which **the job completes** minus the time at which **the job arrived** in the system.

$$T_{turnaround} = T_{completion} - T_{arrival}$$

- **Another metric is fairness.**
  - Performance and fairness are often at odds in scheduling.

First In, First Out (FIFO)

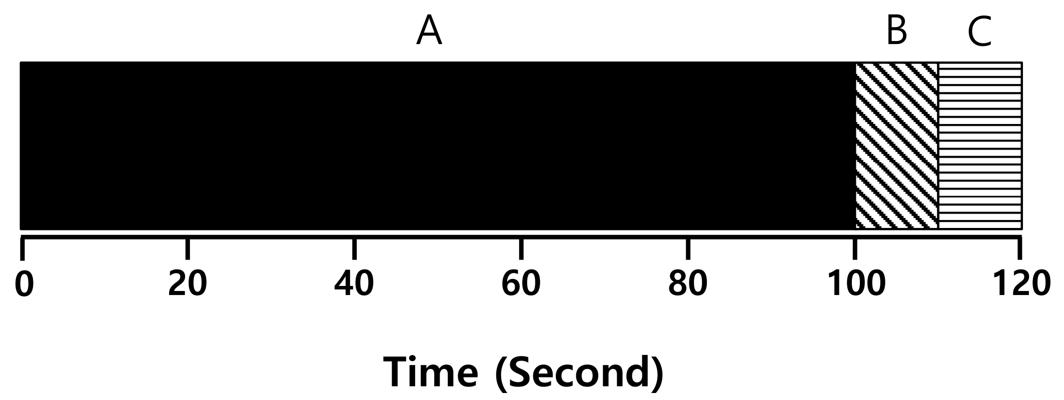
- **First Come, First Served (FCFS)**
  - Very simple and easy to implement
- **Example:**
  - A arrived just before B which arrived just before C.
  - Each job runs for 10 seconds.



$$\boldsymbol{Average\ turnaround\ time = \frac{10 + 20 + 30}{3} = 20\ sec}$$

Why FIFO is not that great? – Convoy effect

- **Let’s relax assumption 1: Each job no longer runs for the same amount of time.**
- **Example:**
  - A arrived just before B which arrived just before C.
  - A runs for 100 seconds, B and C run for 10 each.



$$Average\ turnaround\ time = \frac{100 + 110 + 120}{3} = 110\ sec$$

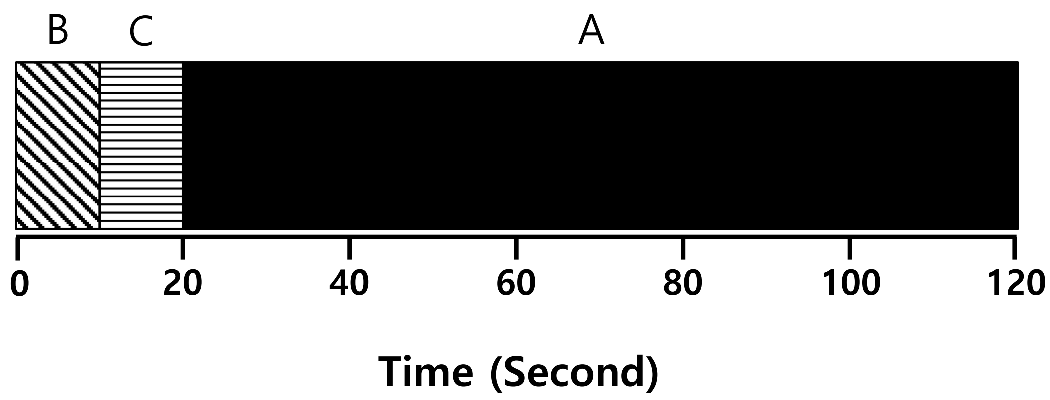
Passing the Tractor

- **Problem with Previous Scheduler:**
  - FIFO: Turnaround time can suffer when short jobs must wait for long jobs

- **New scheduler:**
  - SJF (Shortest Job First)
  - Choose job with smallest *run_time*

Shortest Job First (SJF)

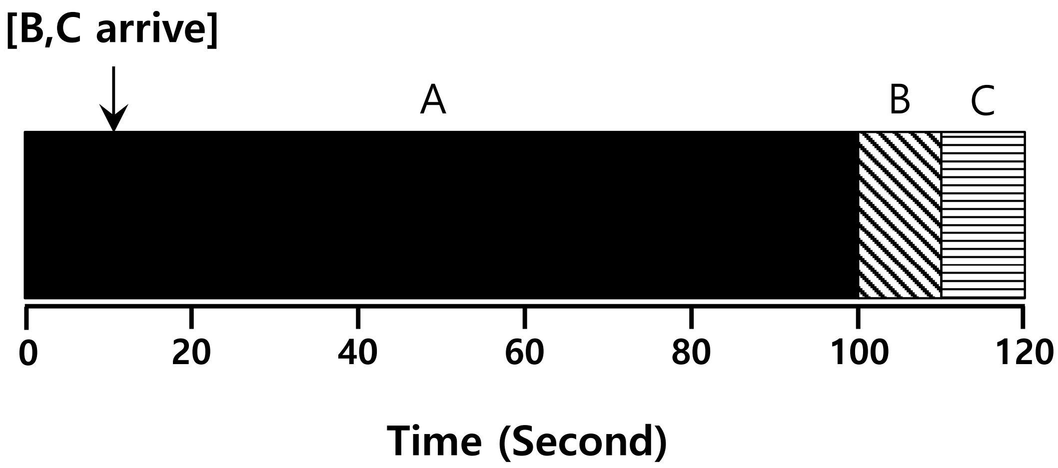
- **Run the shortest job first, then the next shortest, and so on**
  - Non-preemptive scheduler
- **Example:**
  - A arrived just before B which arrived just before C.
  - A runs for 100 seconds, B and C run for 10 each.



$$\textit{Average turnaround time} = \frac{10 + 20 + 120}{3} = 50\ sec$$

SJF with Late Arrivals from B and C

- **Let's relax assumption 2: Jobs can arrive at any time.**
- **Example:**
  - A arrives at t=0 and needs to run for 100 seconds.
  - B and C arrive at t=10 and each need to run for 10 seconds



$$Average\ turnaround\ time = \frac{100 + (110 - 10) + (120 - 10)}{3} = 103.33\ sec$$

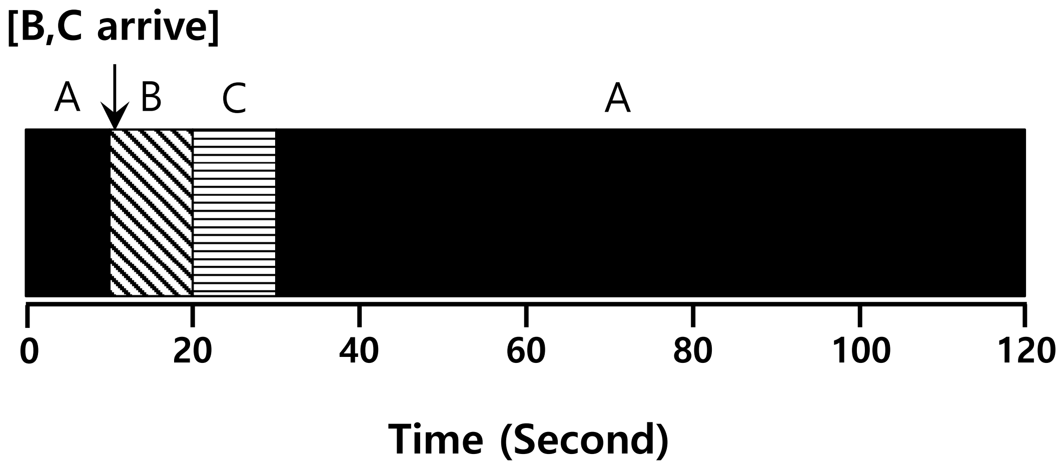
Shortest Time-to-Completion First (STCF)

- **Add preemption to SJF**
  - Also knows as Preemptive Shortest Job First (PSJF)

- **A new job enters the system:**
  - Determine of the remaining jobs and new job
  - Schedule the job which has the lest time left

Shortest Time-to-Completion First (STCF)

- **Example:**
  - A arrives at t=0 and needs to run for 100 seconds.
  - B and C arrive at t=10 and each need to run for 10 seconds



$$\boldsymbol{Average\ turnaround\ time} = \frac{\mathbf{(120-0)+(20-10)+(30-10)}}{\mathbf{3}} = \mathbf{50}\ \boldsymbol{sec}$$

New scheduling metric: Response time

- **The time from when the job arrives to the first time it is scheduled.**

$$T_{response} = T_{firstrun} - T_{arrival}$$

  - STCF and related disciplines are not particularly good for response time.

How can we build a scheduler that is sensitive to response time?

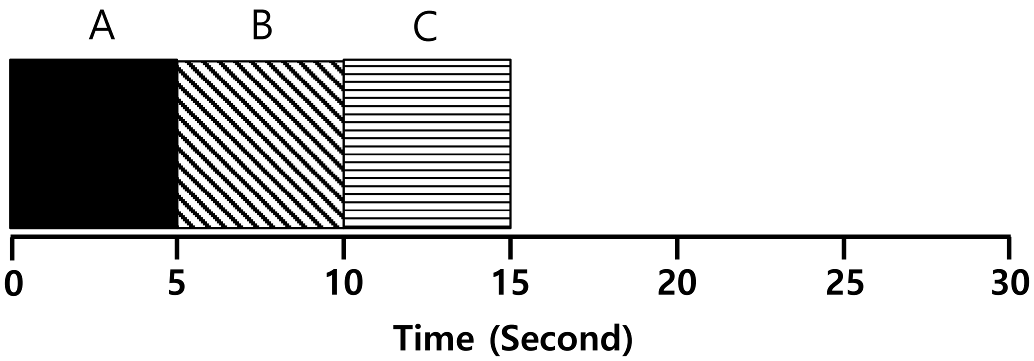
Round Robin (RR) Scheduling

- **Time slicing Scheduling**
  - Run a job for a **time slice** and then switch to the next job in the **run queue** until the jobs are finished.
    - Time slice is sometimes called a scheduling quantum.
  - It repeatedly does so until the jobs are finished.
  - The length of a time slice must be *a multiple of* the timer-interrupt period.

RR is fair, but performs poorly on metrics such as turnaround time

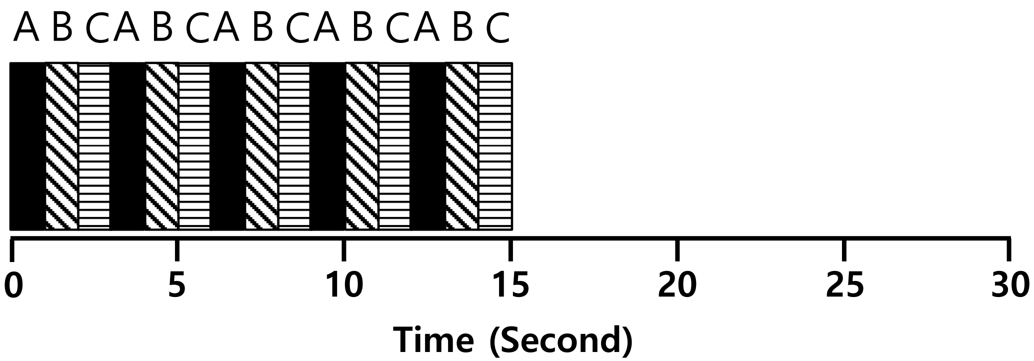
RR Scheduling Example

- A, B and C arrive at the same time.
- They each wish to run for 5 seconds.



SJF (Bad for Response Time)

$$T_{average\ response} = \frac{0+5+10}{3} = 5sec$$



RR with a time-slice of 1sec (Good for Response Time)

$$T_{average\ response} = \frac{0+1+2}{3} = 1sec$$

The length of the time slice is critical.

- **The shorter time slice**
  - Better response time
  - The cost of context switching will dominate overall performance.

- **The longer time slice**
  - Amortize the cost of switching
  - Worse response time

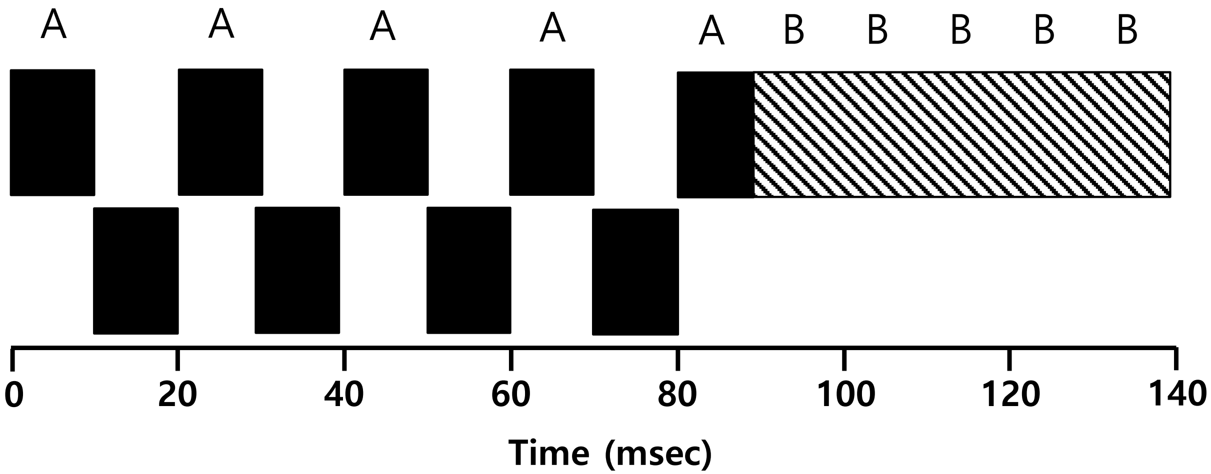
Deciding on the length of the time slice presents a trade-off to a system designer

Incorporating I/O

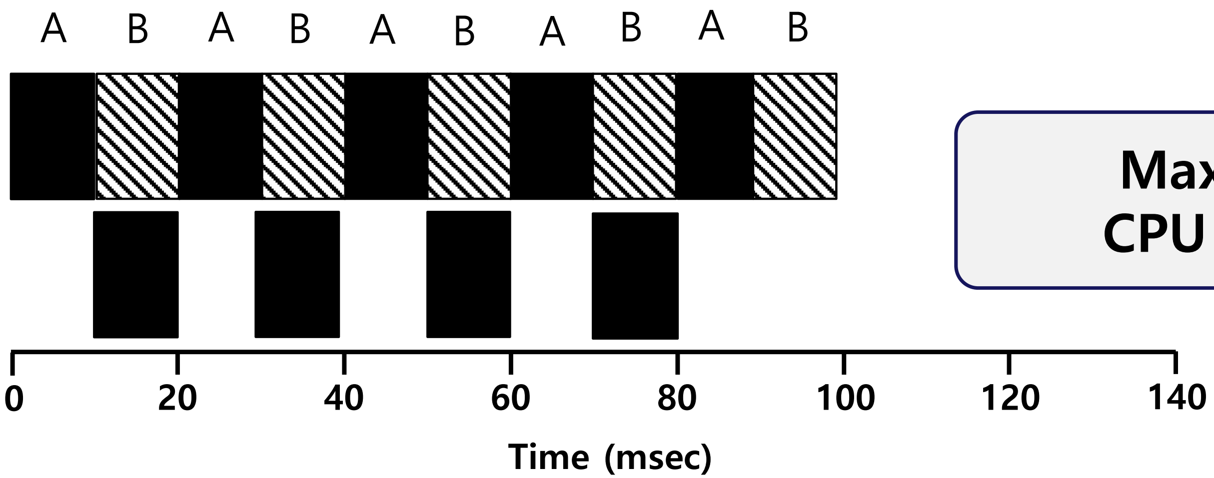
- **Let's relax assumption 3: All programs perform I/O**

- **Example:**
  - A and B need 50ms of CPU time each.
  - A runs for 10ms and then issues an I/O request
    - I/Os each take 10ms
  - B simply uses the CPU for 50ms and performs no I/O
  - The scheduler runs A first, then B after

Incorporating I/O (Cont.)



Poor Use of Resources



Overlap Allows Better Use of Resources

Maximize the CPU utilization

Incorporating I/O (Cont.)

- **When a job initiates an I/O request.**
  - The job is blocked waiting for I/O completion.
  - The scheduler should schedule another job on the CPU.

- **When the I/O completes**
  - An interrupt is raised.
  - The OS moves the process from blocked back to the ready state.

The Multi-Level Feedback Queue

Multi-Level Feedback Queue (MLFQ)

- **A Scheduler that learns from the past to predict the future.**

- **Objective:**
  - Optimize **turnaround time** → Run shorter jobs first
  - Minimize **response time** without *a priori knowledge of job length*.

MLFQ: Basic Rules

- **MLFQ has a number of distinct queues.**
  - Each queues is assigned a different priority level.

- **A job that is ready to run is on a single queue.**
  - A job **on a higher queue** is chosen to run.
  - Use round-robin scheduling among jobs in the same queue

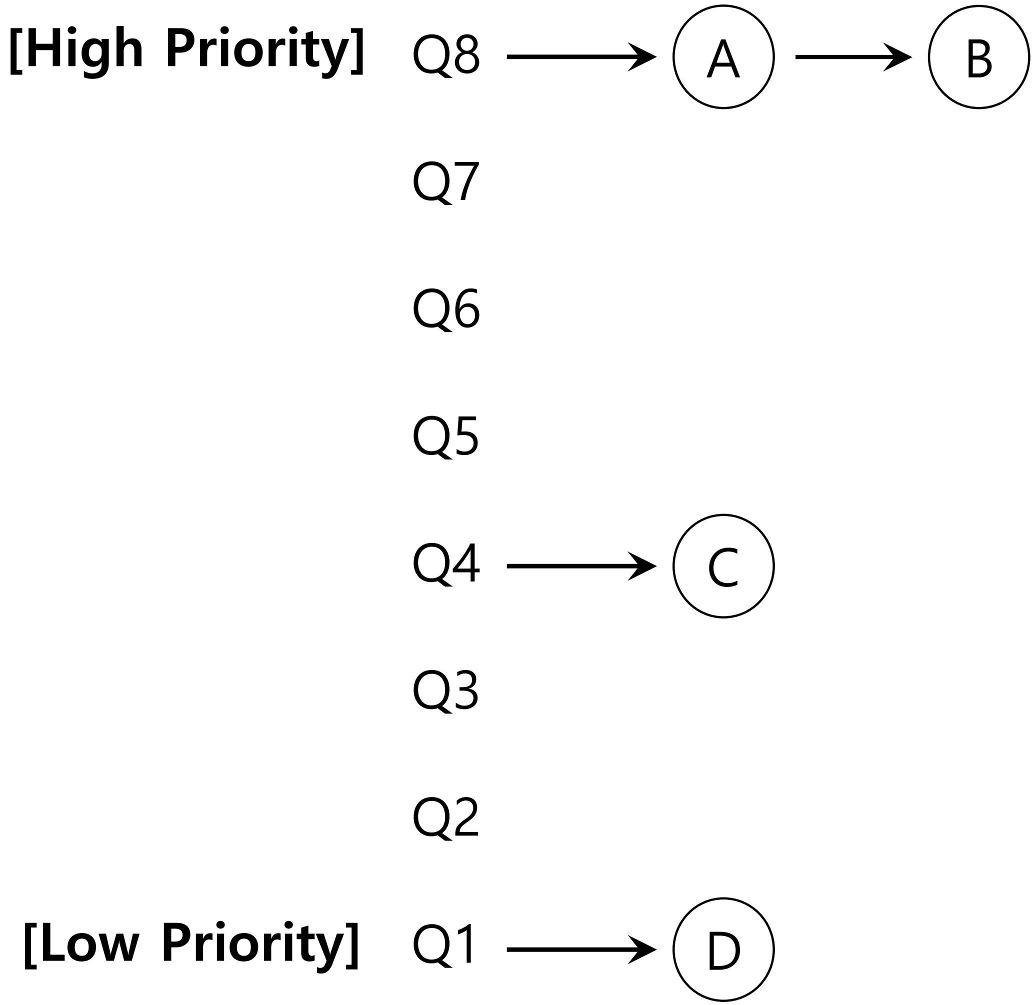
Rule 1: If Priority(A) > Priority(B), A runs (B doesn't).
Rule 2: If Priority(A) = Priority(B), A & B run in RR.

MLFQ: Basic Rules (Cont.)

- **MLFQ varies the priority of a job based on its observed behavior.**

- **Example:**
  - A job repeatedly relinquishes the CPU while waiting IOs → Keep its priority high
  - A job uses the CPU intensively for long periods of time → Reduce its priority.

MLFQ Example



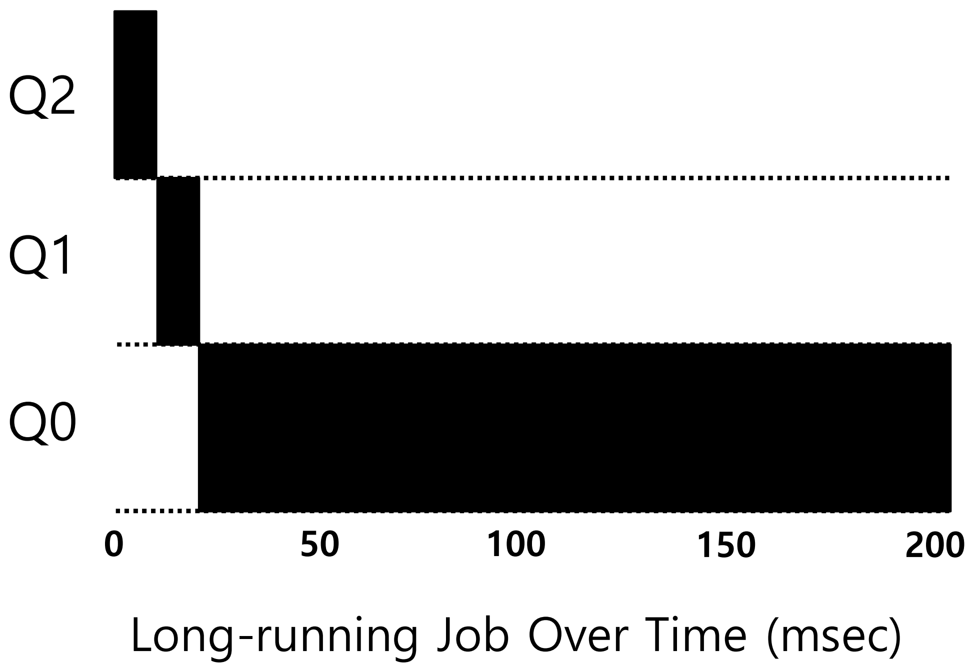
MLFQ: How to Change Priority

- **MLFQ priority adjustment algorithm:**
  - **Rule 3**: When a job enters the system, it is placed at the highest priority
  - **Rule 4a**: If a job uses up an entire time slice while running, its priority is reduced (i.e., it moves down on queue).
  - **Rule 4b**: If a job gives up the CPU before the time slice is up, it stays at the same priority level

In this manner, MLFQ approximates SJF

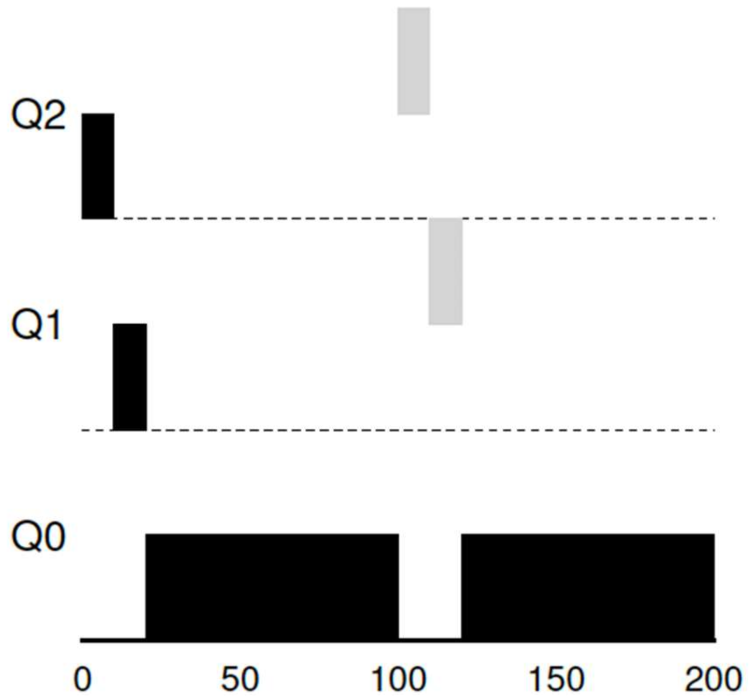
Example 1: A Single Long-Running Job

- **A three-queue scheduler with time slice 10ms**



Example 2: Along Came a Short Job

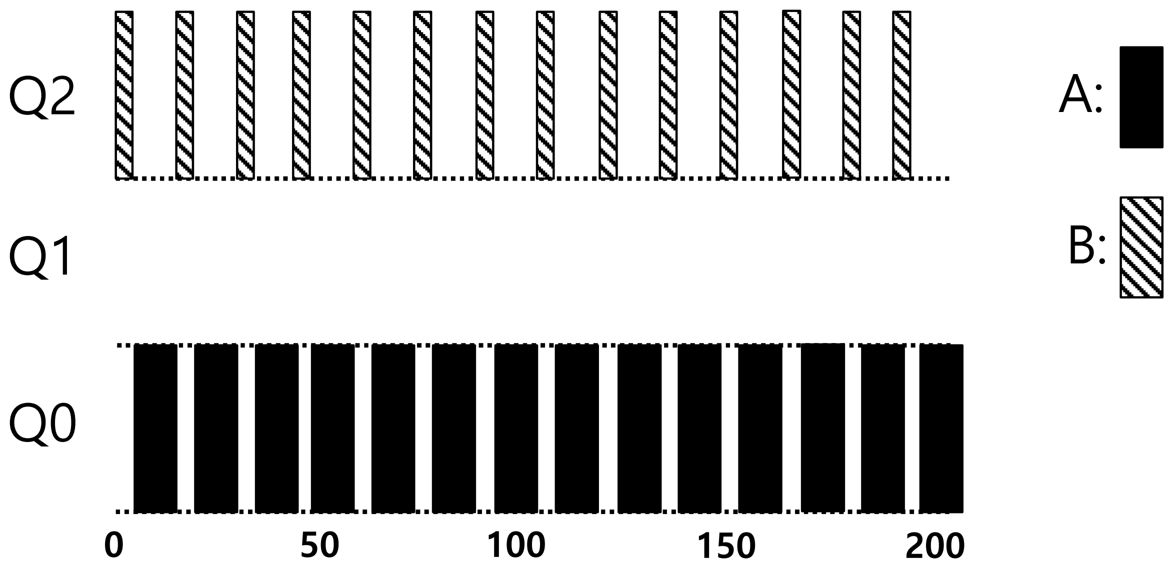
- **Assumption:**
  - **Job A**: A long-running CPU-intensive job
  - **Job B**: A short-running interactive job (20ms runtime)
  - A has been running for some time, and then B arrives at time T=100.



Along Came An Interactive Job (msec)

Example 3: What About I/O?

- **Assumption:**
  - **Job A**: A long-running CPU-intensive job
  - **Job B**: An interactive job that need the CPU only for 1ms before performing an I/O



A Mixed I/O-intensive and CPU-intensive Workload (msec)

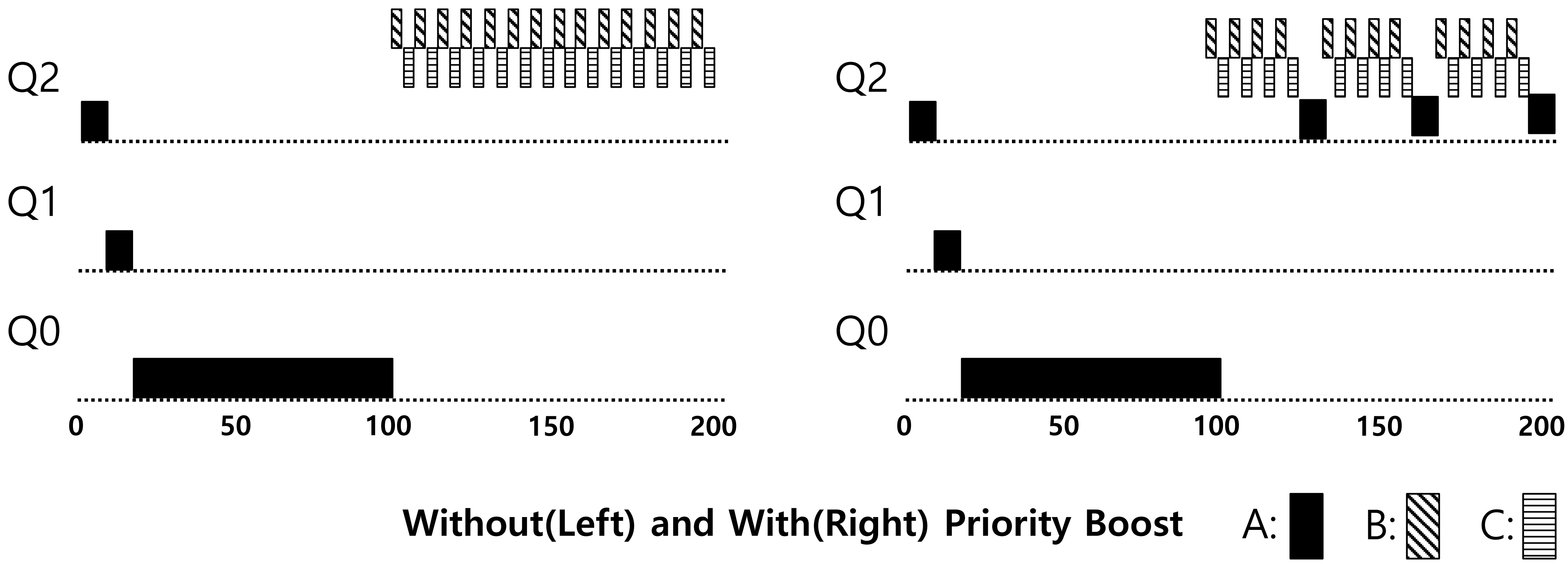
The MLFQ approach keeps an interactive job at the highest priority

Problems with the Basic MLFQ

- **Starvation**
  - If there are “too many” interactive jobs in the system.
  - Lon-running jobs will never receive any CPU time.
- **Game the scheduler**
  - After running 99% of a time slice, issue an I/O operation.
  - The job gains a higher percentage of CPU time.
- **A program may change its behavior over time.**
  - CPU bound process → I/O bound process

The Priority Boost

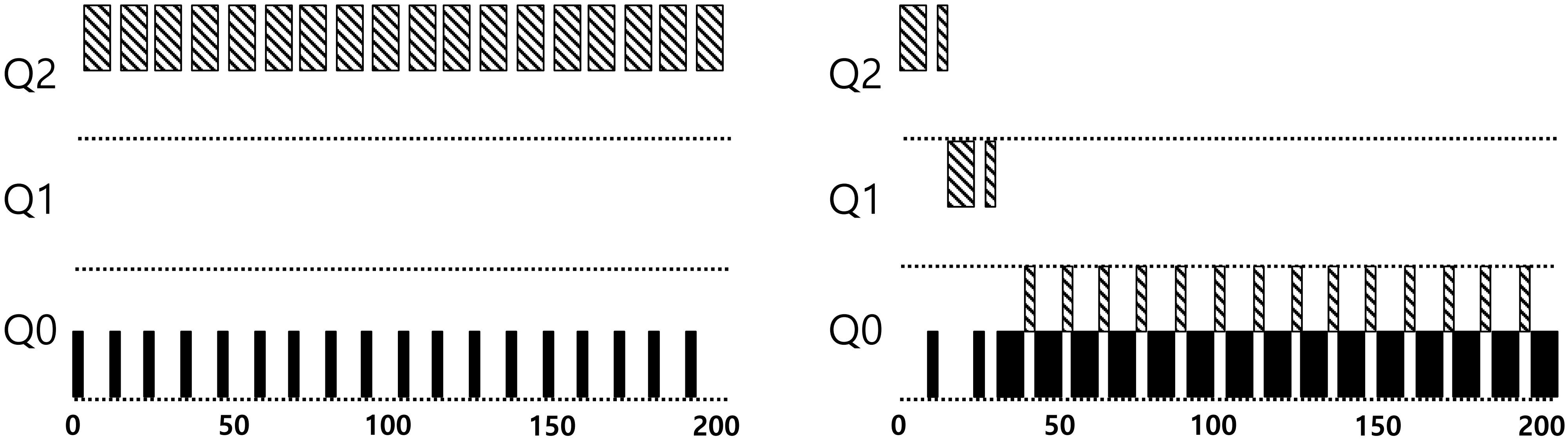
- **Rule 5: After some time period S, move all the jobs in the system to the topmost queue.**
  - Example:
    - A long-running job(A) with two short-running interactive job(B, C)



Without(Left) and With(Right) Priority Boost

Better Accounting

- **How to prevent gaming of our scheduler?**
- **Solution:**
  - **Rule 4** (Rewrite Rules 4a and 4b): Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), **its priority is reduced**(i.e., it moves down on queue).

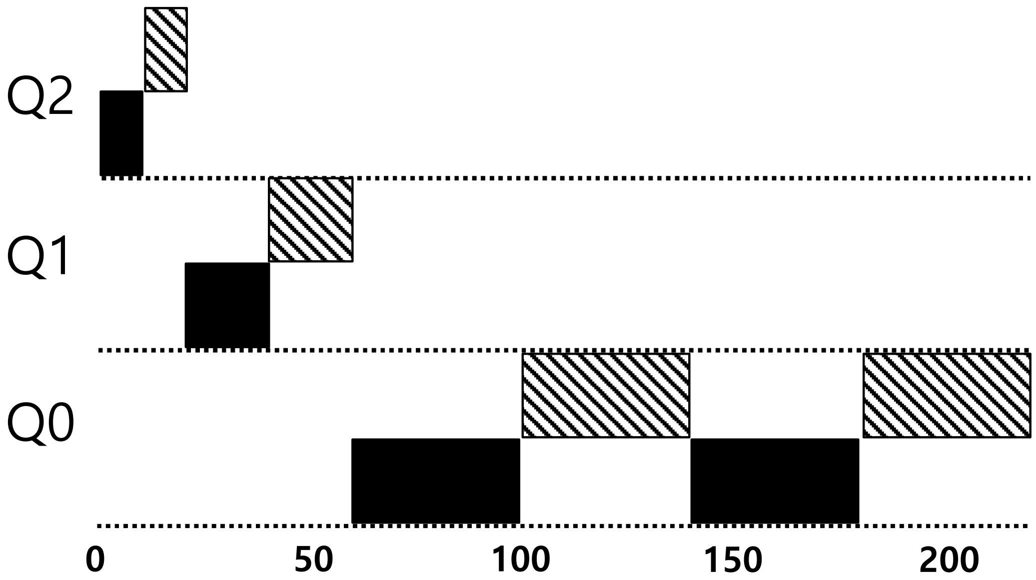


Without(Left) and With(Right) Gaming Tolerance

Tuning MLFQ And Other Issues

Lower Priority, Longer Quanta

- **The high-priority queues → Short time slices**
  - E.g., 10 or fewer milliseconds
- **The Low-priority queue → Longer time slices**
  - E.g., 100 milliseconds



Example) 10ms for the highest queue, 20ms for the middle, 40ms for the lowest

The Solaris MLFQ implementation

- **For the Time-Sharing scheduling class (TS)**
  - 60 Queues
  - Slowly increasing time-slice length
    - The highest priority: 20msec
    - The lowest priority: A few hundred milliseconds
  - Priorities boosted around every 1 second or so.

MLFQ: Summary

- **The refined set of MLFQ rules:**
  - **Rule 1:** If Priority(A) > Priority(B), A runs (B doesn't).
  - **Rule 2:** If Priority(A) = Priority(B), A & B run in RR.
  - **Rule 3:** When a job enters the system, it is placed at the highest priority.
  - **Rule 4:** Once a job uses up its time allotment at a given level (regardless of how many times it has given up the CPU), its priority is reduced(i.e., it moves down on queue).
  - **Rule 5:** After some time period S, move all the jobs in the system to the topmost queue.

- **Beauty of MLFQ**
  - It does not require prior knowledge on the CPU usage of a process.

Multiprocessor Scheduling

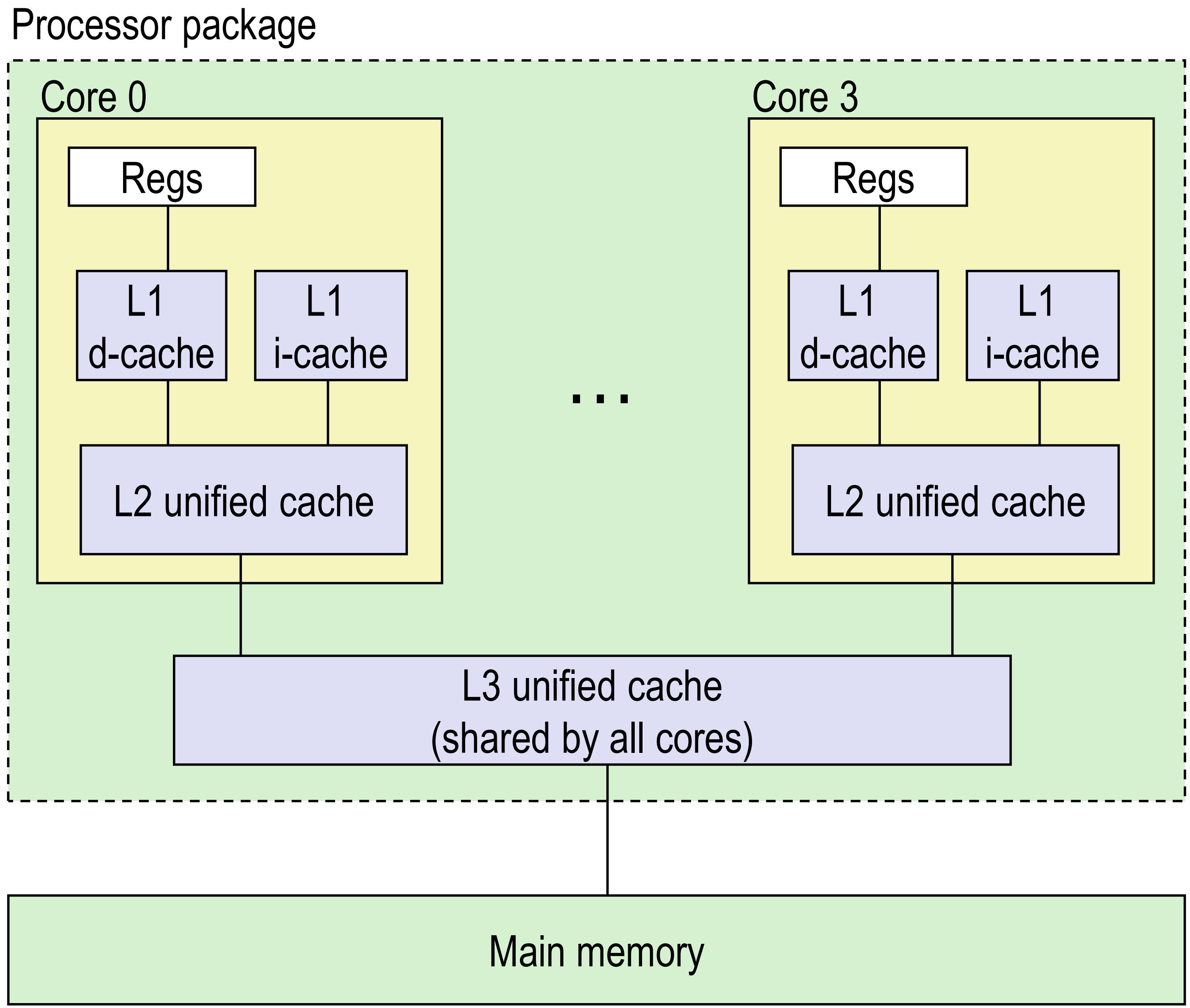
Multiprocessor Scheduling

- **The rise of the multicore processor is the source of multiprocessor-scheduling proliferation.**
  - **Multicore**: Multiple CPU cores are packed onto a single chip.

- **Adding more CPUs does not make that single application run faster. → You'll have to rewrite application to run in parallel, using threads.**

How to schedule jobs on Multiple CPUs?

Example: Multicore CPU Cache Hierarchy



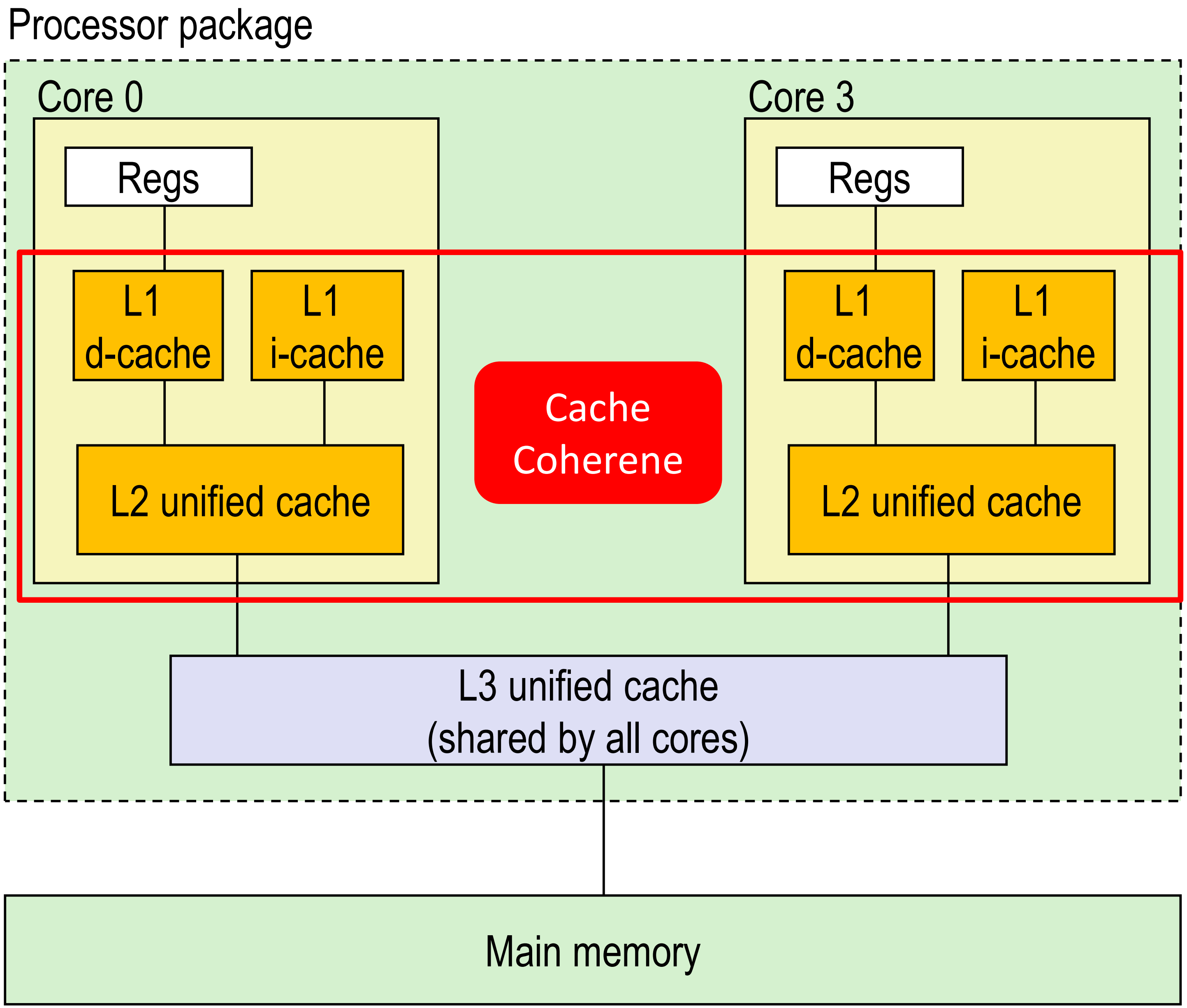
L1 i-cache and d-cache:
32 KB, 8-way,
Access: 4 cycles

L2 unified cache:
256 KB, 8-way,
Access: 10 cycles

L3 unified cache:
8 MB, 16-way,
Access: 40-75 cycles

Block size: 64 bytes for al
l caches.

Example: Multicore CPU Cache Hierarchy



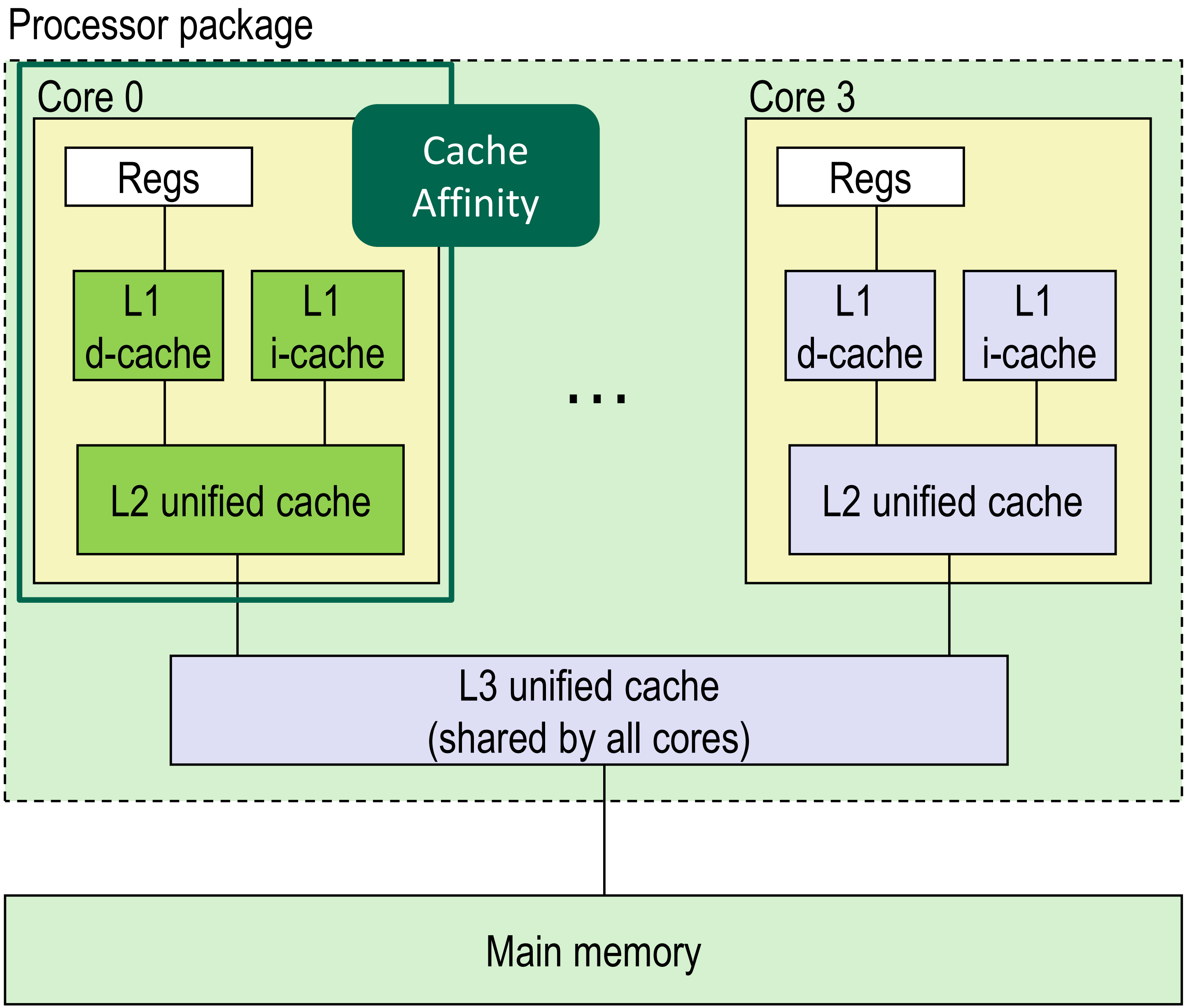
L1 i-cache and d-cache:
32 KB, 8-way,
Access: 4 cycles

L2 unified cache:
256 KB, 8-way,
Access: 10 cycles

L3 unified cache:
8 MB, 16-way,
Access: 40-75 cycles

Block size: 64 bytes for al
l caches.

Example: Multicore CPU Cache Hierarchy



L1 i-cache and d-cache:
32 KB, 8-way,
Access: 4 cycles

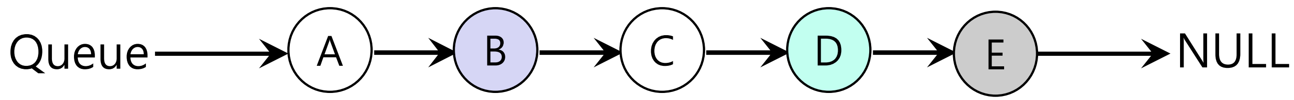
L2 unified cache:
256 KB, 8-way,
Access: 10 cycles

L3 unified cache:
8 MB, 16-way,
Access: 40-75 cycles

Block size: 64 bytes for al
l caches.

Single Queue Multiprocessor Scheduling (SQMS)

- **Put all jobs that need to be scheduled into a single queue.**
  - Each CPU simply picks the next job from the globally shared queue.



Possible job scheduler across CPUs:

CPU0	A	E	D	C	B	... (repeat) ...
CPU1	B	A	E	D	C	... (repeat) ...
CPU2	C	B	A	E	D	... (repeat) ...
CPU3	D	C	B	A	E	... (repeat) ...

Single Queue Multiprocessor Scheduling (SQMS)

- **Put all jobs that need to be scheduled into a single queue.**
  - Each CPU simply picks the next job from the globally shared queue.

Queue → A → B → C → D → E → NULL

Synchronization Overhead:

- Multiple CPUs attempting to access and modify the same queue concurrently create **race conditions**.
- To ensure data integrity, **locks** (mutexes) must be used.

Possible job scheduler across CPUs:

CPU0	A	E	D	C	B	... (repeat) ...
CPU1	B	A	E	D	C	... (repeat) ...
CPU2	C	B	A	E	D	... (repeat) ...
CPU3	D	C	B	A	E	... (repeat) ...

Single Queue Multiprocessor Scheduling (SQMS)

- **Put all jobs that need to be scheduled into a single queue.**
  - Each CPU simply picks the next job from the globally shared queue.

Queue → A → B → C → D → E → NULL

Synchronization Overhead:
- Multiple CPUs attempting to access and modify the same queue concurrently create **race conditions**.
- To ensure data integrity, **locks** (mutexes) must be used.

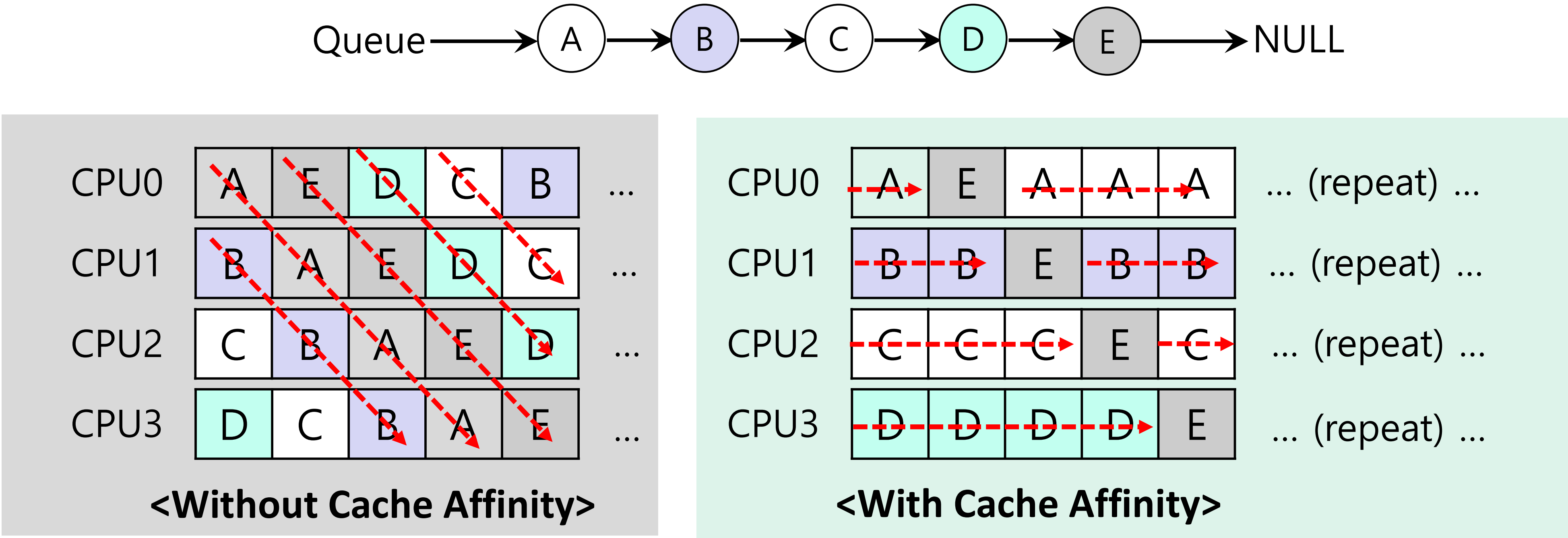
Possible job scheduler across CPUs:

CPU0	A	E	D	C	B	... (repeat) ...
CPU1	B	A	E	D	C	... (repeat) ...
CPU2	C	B	A	E	D	... (repeat) ...
CPU3	D	C	B	A	E	... (repeat) ...

Frequent Process Migration:
- A process can run on CPU0, but might be executed on CPU1 for its next turn.

Cache Misses and Perf. Degradation:
- When the process moves to a new CPU (CPU1), its cached data is not present in CPU1's cache.

Scheduling Example with Cache Affinity



- Preserving affinity for most
  - Jobs A through D are not moved across processors.
  - Only job e Migrating from CPU to CPU.
- Implementing such a scheme can be **complex**.

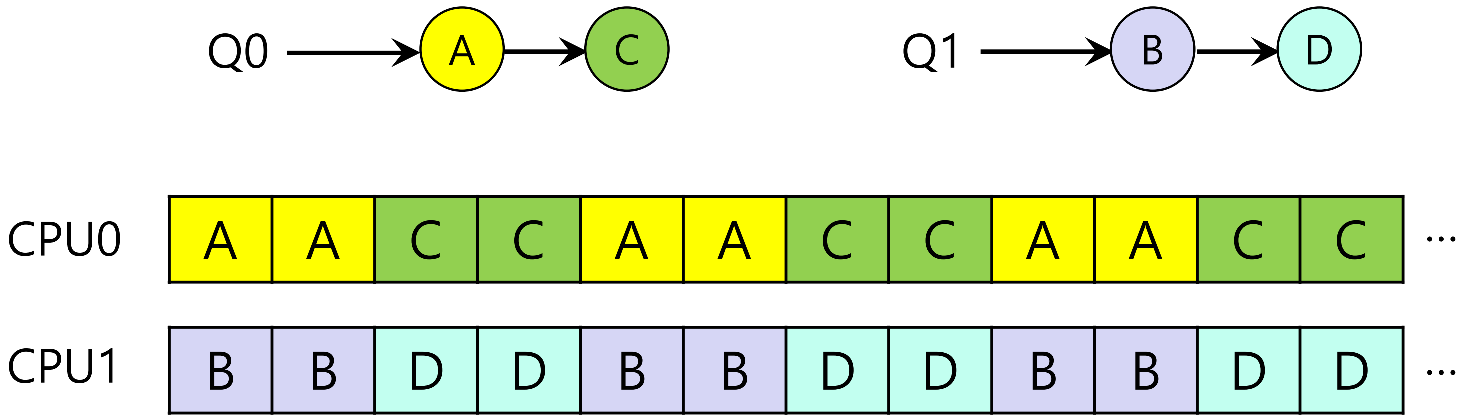
Multi Queue Multiprocessor Scheduling (MQMS)

- **MQMS provides each CPU with its own private scheduling queue.**
  - When a job enters the system, it is placed on exactly one scheduling queue.

- **Design Benefits**
  - **Independent Queue Operation**: Each CPU manages and schedules tasks exclusively from its own local queue.
  - **No Synchronization Needed**: Because the queues are independent, CPUs do not need to coordinate access to their respective queues using locks.
  - **Reduced Overhead & Improved Performance**: The removal of lock acquisition and release overhead significantly improves system scalability and overall performance.

MQMS Example

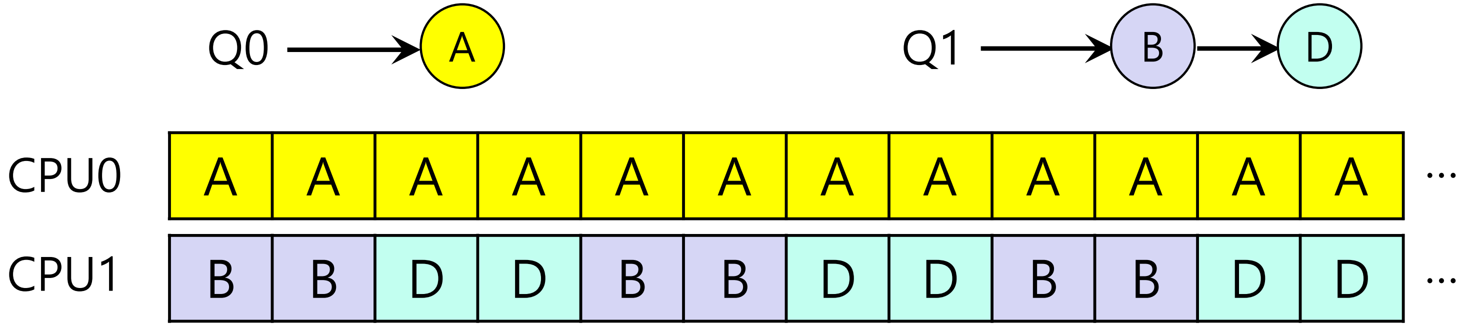
- **With Round Robin, the system might produce a schedule that looks like this:**



MQMS achieves better performance by reducing inter-core communication (scalability) and maximizing local cache utilization (cache affinity).

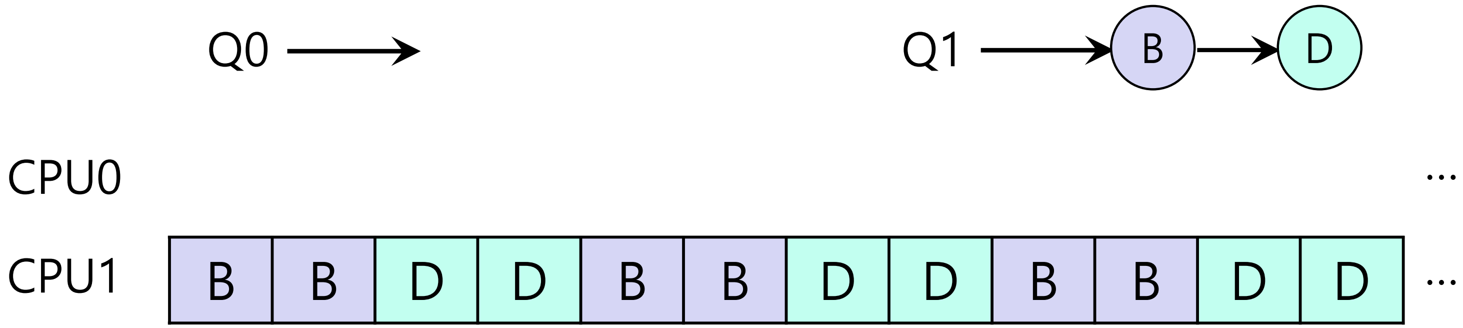
Load Imbalance Issue of MQMS

- **After job C in Q0 finishes:**



`A` gets twice as much CPU as `B` and `D`.

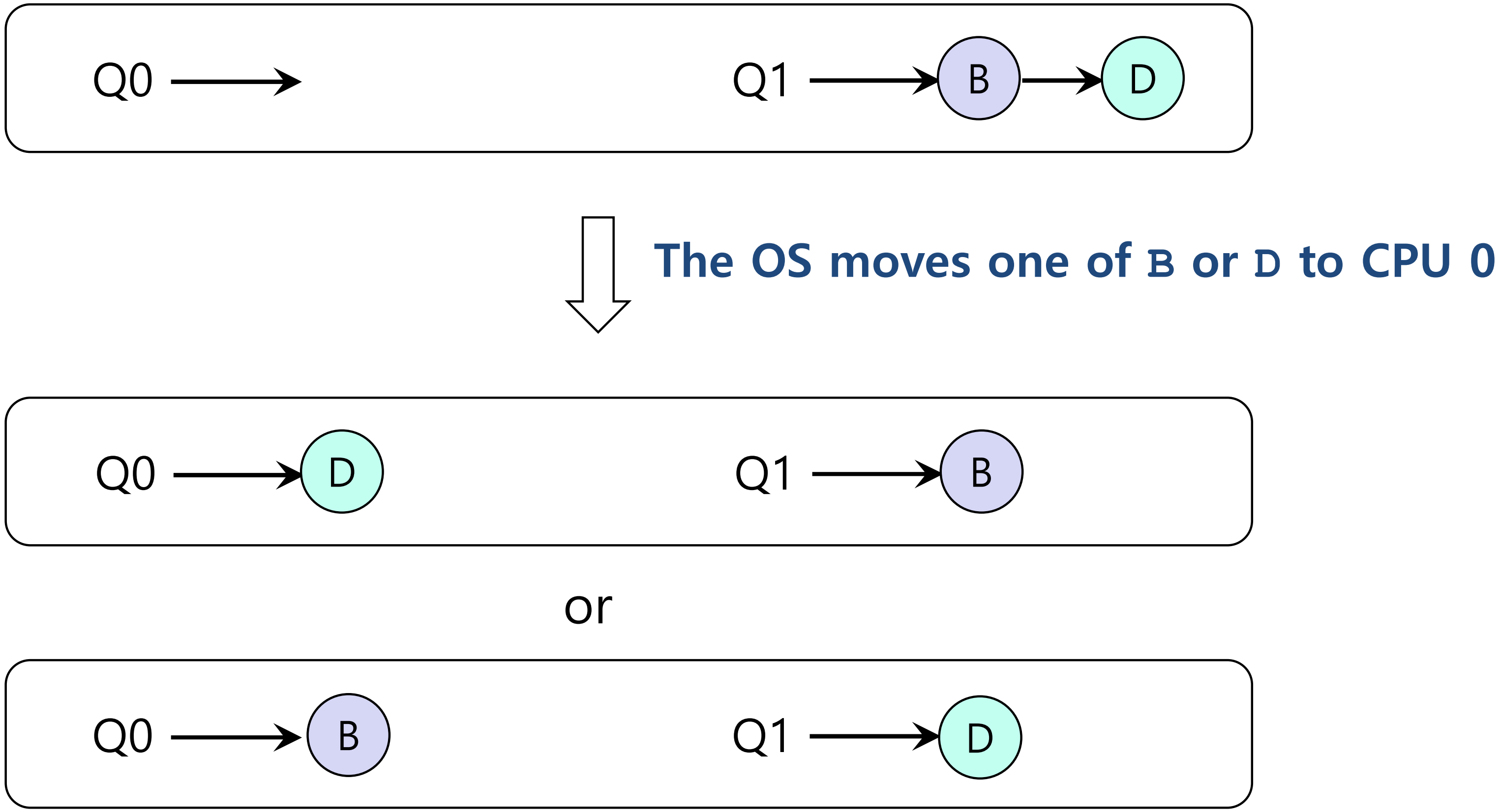
- **After job A in Q0 finishes:**



CPU0 will be left idle!

How to deal with load imbalance?

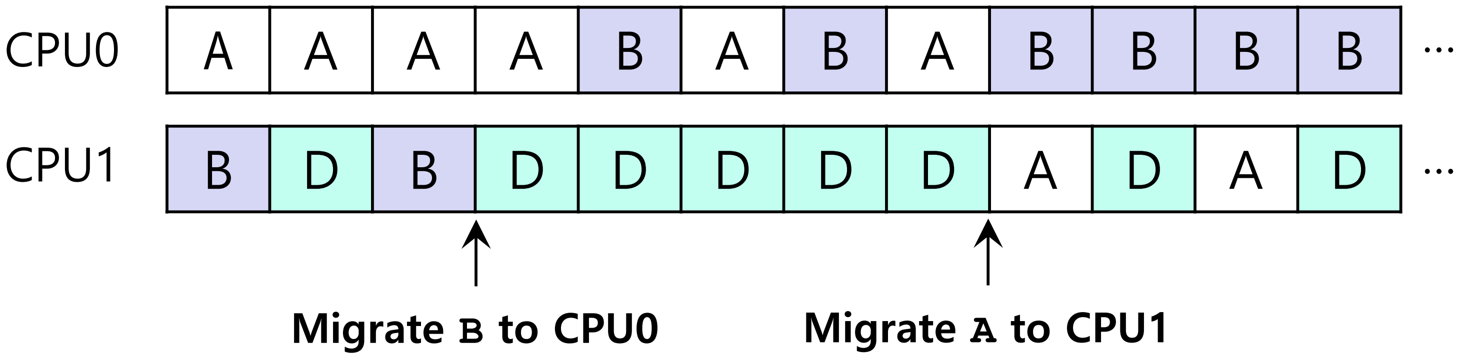
- **The answer is to move jobs (Migration).**
  - Example:



How to deal with load imbalance? (Cont.)

- **A more tricky case:**

- **A possible migration pattern:**
  - Keep switching jobs



CPU Scheduling: Summary

Type	Algorithm
Non-Preemptive	**FCFS** – simple, causes convoy effect
	SJF – shortest job first, best avg. turnaround
Preemptive	**STCF** – run job with least time left
	RR – time slicing for fairness
	MLFQ – adaptive priority, learns from behavior

- **Goals**
  - Efficiency: Maximize CPU utilization
  - Fairness: Equal share among processes
  - Responsiveness: Minimize response time
- **MLFQ Highlights**
  - Start at high priority, demote on long use
  - Promote periodically to avoid starvation
  - Mixes fairness (RR) + efficiency (SJF)
- **Multiprocessor Scheduling**
  - Cache affinity: keep process on same CPU
  - Single Queue (SQMS): simple but low scalability
  - Multi-Queue (MQMS): per-CPU queues, use work stealing for load balance